

Дополнительные главы дискретной математики:

Алгоритмы и структуры данных

Лекция 6. Кучи, очереди с приоритетами и пирамидальная сортировка

Ключарёв Пётр Георгиевич
pgkl@yandex.ru
twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

2013 г.

Двоичная куча

Двоичная куча

Двоичная куча — это двоичное дерево, хранящееся в массиве. Для двоичной кучи должны выполняться специальные свойства упорядоченности.

Связь между деревом и массивом

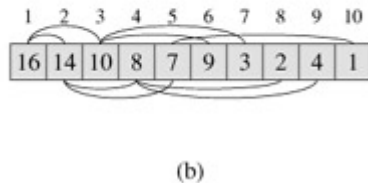
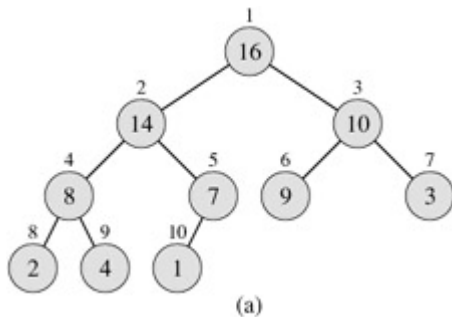
Каждая вершина дерева соответствует элементу массива:

$A[1], \dots, A[\text{heapsize}]$. Корень имеет индекс 1.

У вершины i есть родитель $\lfloor i/2 \rfloor$ и дети $2i$ и $2i + 1$.

Параметры: длина массива $length$ и размер кучи: heap-size .

Пример кучи



Обозначения

Обозначения

$Parent(i)$ — родитель i -ой вершины (индекс).

$Left(i)$ — левый потомок i -ой вершины (индекс).

$Right(i)$ — правый потомок i -ой вершины (индекс).

Высота

Все уровни в дереве заполнены полностью. Поэтому высота дерева:
 $\Theta(\log n)$

Основное свойство кучи

Для каждой вершины с индексом i , кроме корня (при $2 \leq i \leq \text{heapsize}$), выполняется: $A[Parent(i)] \geq A[i]$ (Если наоборот — выполняется $A[Parent(i)] \leq A[i]$, то куча называется минимальной кучей — MinHeap).

Следовательно, значение потомка не превосходит значения предка.

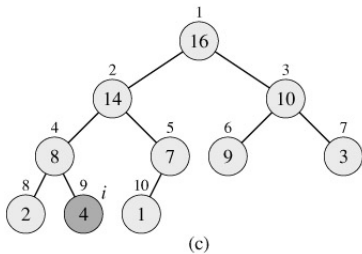
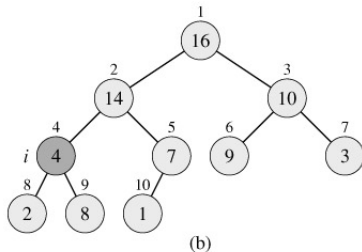
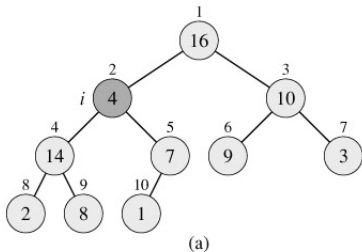
Heapify

Heapify — вспомогательный алгоритм, получающий на вход массив A и индекс i и делающий так, чтобы основное свойство кучи для поддерева с корнем i выполнялось. Сложность $O(\log n)$. Предполагается, что поддеревья с корнями $Left(i)$ и $Right(i)$ обладают основным свойством кучи.

Алгоритм 1 Heapify

```
1: procedure HEAPIFY( $A, i$ )
2:    $l := Left(i); r := Right(i)$ 
3:   if  $l \leq heapsize$  и  $A[l] > A[i]$  then
4:      $largest := l$ 
5:   else
6:      $largest := i$ 
7:   if  $r \leq heapsize$  и  $A[r] > A[largest]$  then  $largest := r$ 
8:   if  $largest \neq i$  then
9:     обменять  $A[i]$  и  $A[largest]$ 
10:    HEAPIFY( $A, largest$ )
```

Пример работы алгоритма Heapify



Построение кучи

Алгоритм 2 Построение кучи

```
1: procedure BUILDHEAP(A)  
2:   heapsize := length(A)  
3:   for i :=  $\lfloor \text{length}(A)/2 \rfloor$  downto 1 do  
4:     HEAPIFY(A, i)
```

Сложность

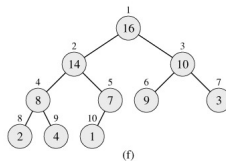
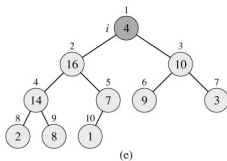
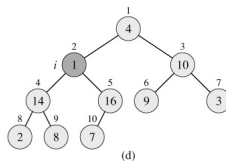
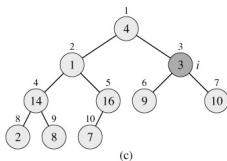
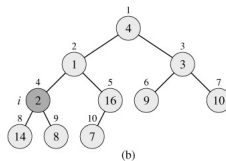
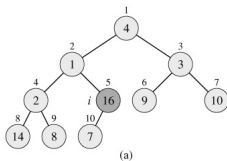
Время работы процедуры HEAPIFY(*A*, *i*) пропорционально высоте вершины *i*. Заметим, что если *t* — высота дерева, то $n < 2^{t+1}$.

Число вершин высоты *h* не превышает 2^{t-h} . Тогда время работы BUILDHEAP можно оценить как:

$$\sum_{h=0}^t 2^{t-h} O(h) = O\left(2^t \sum_{h=0}^t \frac{h}{2^h}\right) = O(2^t \cdot 2) = O(n).$$

Пример работы алгоритма BUILDHEAP

A [4 1 3 2 16 9 10 14 8 7]



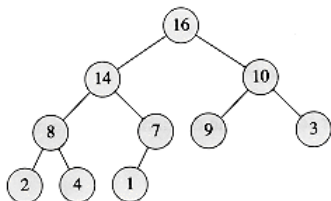
Добавление элемента

Алгоритм 3 Добавление элемента

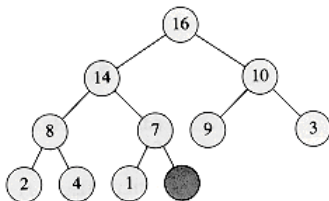
```
1: procedure HEAPINSERT( $A, key$ )  
2:    $heapsize := heapsize + 1$   
3:    $i := heapsize$   
4:   while  $i > 1$  и  $A[Parent(i)] < key$  do  
5:      $A[i] := A[PARENT(i)]$   
6:      $i := PARENT(i)$   
7:    $A[i] := key$ 
```

Время работы: $O(\log n)$

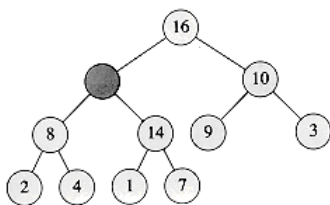
Пример вставки элемента с ключом 15 (HEAPINSERT)



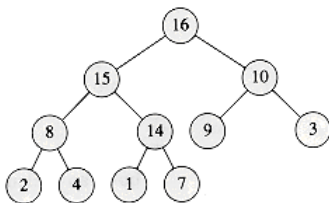
(a)



(b)



(c)



(d)

Максимальный элемент

Алгоритм 4 Возвращает максимальный элемент

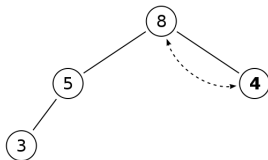
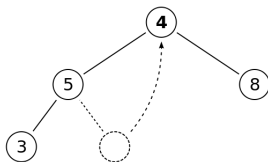
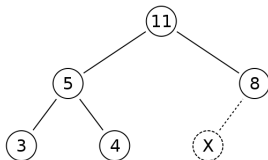
```
1: function MAXIMUM( $A$ )  
2:    $max := A[1]$   
3:   return  $max$ 
```

Алгоритм 5 Удаление максимального элемента

```
1: function HEAPEXTRACTMAX( $A$ )  
2:   if  $heapsize \leq 0$  then  
3:     error «куча пуста»  
4:    $max := A[1]$   
5:    $A[1] := A[heapsize]$   
6:    $heapsize := heapsize - 1$   
7:   HEAPIFY( $A, 1$ )  
8:   return  $max$ 
```

Время работы: $O(\log n)$

Пример работы алгоритма HEAPEXTRACTMAX



Очередь с приоритетами

Очередь с приоритетами

Очередь с приоритетами (priority queue) — это множество S , элементами которого являются пары $\langle key, \alpha \rangle$, где key — число, определяющее приоритет элемента, а α — связанная с ним информация. Для простоты изложения, мы будем считать, что элементами множества являются только ключи.

При извлечении элемента из очереди, каждый раз извлекается элемент с наибольшим приоритетом.

Реализация очереди с приоритетами

Реализация основана на куче. Для добавления элемента используется алгоритм HEAPINSERT. Для извлечения элемента — алгоритм HEAPEXTRACTMAX.

Обе эти операции требуют времени $O(\log n)$.

Пирамидальная сортировка

Алгоритм 6 Пирамидальная сортировка

```
1: procedure HEAPSORT( $A$ )  
2:   BUILDHEAP( $A$ )  
3:   for  $i := \text{length}(A)$  downto 2 do  
4:     Поменять  $A[1]$  и  $A[i]$ .  
5:      $\text{heapsize} := \text{heapsize} - 1$   
6:     HEAPIFY( $A, 1$ )
```

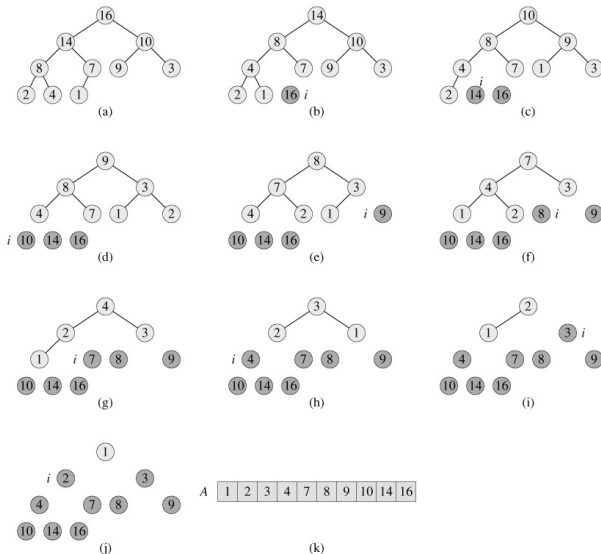
Сложность

Построение дерева требует времени $O(n)$.

В цикле n раз производится обмен корня с последним элементом и вызов HEAPIFY, что занимает время $O(\log n)$. Таким образом, сложность пирамидальной сортировки составляет:

$$O(n \log n)$$

Пример работы пирамидальной сортировки



Упражнения

- 1 Является ли кучей массив $\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$?
- 2 Доказать, что сложность процедуры HEAPIFY равна $\Theta(\log n)$.
- 3 Покажите как работает процедура HEAPIFY($A, 3$) для массива $\langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0 \rangle$
- 4 Покажите, как работает процедура BUILD-MAX-HEAP для массива $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$.
- 5 Покажите, как работает процедура HEAPSORT для массива $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$.
- 6 Пусть исходный массив A уже отсортирован в порядке возрастания. Каково будет время сортировки с помощью кучи? А если массив был отсортирован в порядке убывания?
- 7 Докажите, что время работы процедуры HEAPSORT составляет $\Omega(n \log n)$.

И еще упражнения

- 1 Объясните, как реализовать обычную очередь (first-in, first-out) и стек на базе очереди с приоритетами.
- 2 Реализуйте операцию удаления элемента с индексом i из кучи. Время работы $O(\log n)$.
- 3 Докажите, что куча из n элементов содержит не более $\lceil n/2^{h+1} \rceil$ вершин высоты h .